PriX: Compiling Private Transformers via Whole-Graph XLA Fusion in Secure Multi-Party Computation

Mehmet Ugurbil
mugurbil@gmail.com
https://github.com/mugurbil/prix

Abstract

Private inference of Transformer and Large Language Models (LLMs) via Secure Multi-Party Computation (MPC) faces a fundamental systems bottleneck: existing frameworks such as CrypTen execute secret-shared protocols over $\mathbb{Z}_{2^{64}}$ via eager, operator-by-operator dispatch in PyTorch. While recent algorithmic advances—Wavelet-encoded Look-Up Tables (CuRL), 1-round Fourier-series activations and ODE Softmax (SHAFT), and hybrid plaintext evaluators (Fission)—substantially reduce communication rounds, their reference implementations remain fragmented across incompatible legacy PyTorch forks and suffer from severe kernel-launch and host–device synchronization overhead on modern accelerators.

We present **PriX**, an open-source JAX/XLA framework for private Transformer inference that unifies all four MPC approximation paradigms (**CrypTen**, **SHAFT**, **CuRL**, and **Fission**) under a single researcher-friendly API while compiling local cryptographic ring operations into fused XLA kernels. PriX introduces four systems optimizations: (1) *exact 4-limb FP64 matrix multiplication* over $\mathbb{Z}_{2^{64}}$, leveraging accelerator floating-point tensor units without 53-bit mantissa overflow; (2) *whole-graph* `@jax.jit` *fusion* of multi-round MPC circuits, collapsing Kogge–Stone carry-lookahead rounds, multi-table wavelet LUT gathers, and Fourier/ODE recurrences into single compiled XLA kernels; (3) *batched single-round Beaver triple opening and wrap-count protocols*; and (4) *lock-free shared-memory and batched host-staged IPC* for multi-party collectives. Across 60 head-to-head benchmarks spanning 10 MPC primitives and 5 Transformer architectures (`BertTiny`, `BertBase`, `GPT2`, `Llama`, and `ModernBert`) on both CPU and NVIDIA Tesla T4 GPU, PriX outperforms the PyTorch CrypTen baseline on **100% of benchmarks (60/60)**, achieving **1.47**×–**6.26**× end-to-end LLM speedups on CPU, **1.97**×–**3.09**× LLM speedups on GPU, and up to **10.30**× speedups on individual nonlinearities.

1 Introduction

Deploying Transformer-based Large Language Models (LLMs) [16] on sensitive user prompts or proprietary model weights requires cryptographic privacy guarantees. Secure Multi-Party Computation (MPC) over the machine ring $\mathbb{Z}_{2^{64}}$ [11, 12, 8, 6] enables two or more distrusting parties to jointly evaluate neural networks on secret-shared inputs without revealing either the client's prompt or the server's parameters.

Over the past five years, CrypTen [8] has served as the foundational Python research platform for machine-learning-oriented MPC. Building upon CrypTen, a wave of recent cryptographic protocol innovations has dramatically improved the accuracy and round complexity of Transformer nonlinearities (GeLU, SiLU, Softmax, LayerNorm, RMSNorm):

- **Wavelet Approximations and CuRL** [5, 13, 14]: Wavelet-based cryptographic function approximation—pioneered across Fully Homomorphic Encryption in **Ripple** [5], mixed-mode MPC in **Wave Hello to Privacy** [13], and private LLM evaluation in **CuRL** [14]—compresses lookup tables via Haar and Biorthogonal-2.2 wavelet transforms paired with probabilistic truncation (TruncPr) [4], substantially reducing LUT communication and memory footprints while preserving high numerical fidelity.

- **SHAFT** [7] introduced 1-round trigonometric-triple Fourier-series evaluation for GeLU, SiLU, Erf, Sigmoid, and Tanh, alongside a constant-round Forward-Euler Ordinary Differential Equation (ODE) solver for Softmax.

- **Fission** [15] introduced a hybrid permute-and-reveal architecture that offloads elementwise nonlinearities to a non-colluding plaintext evaluator party.

Despite these algorithmic breakthroughs, MPC research for Transformers faces two critical systems obstacles:

1. **Ecosystem Fragmentation and Legacy Dependencies:** Original CrypTen has been archived since 2023 and is pinned to legacy PyTorch 1.9 and Python 3.7, while CuRL, SHAFT, and Fission reside in separate, mutually incompatible forks. Researchers conducting ablation studies across MPC approximation paradigms must port models between distinct codebases.

2. **Eager Dispatch and Accelerator Underutilization:** In eager PyTorch, every elementary bitwise shift, mask, and ring addition in an MPC circuit dispatches an individual CPU or CUDA kernel. For example, a single 64-bit Kogge–Stone comparison [9] executes 6 tree rounds of bitwise XOR and AND operations; in eager mode, this triggers over 18 tiny kernel launches interleaved with host-side Python control flow. Furthermore, because CUDA lacks native tensor-core support for 64-bit integer (`int64`) matrix multiplication, PyTorch's CrypTen forks decompose 64-bit matrices into four 16-bit limbs and execute *16 separate FP64* `torch.matmul` *kernels* per private matrix multiplication. Consequently, GPU execution in existing CrypTen forks is heavily bottlenecked by launch overhead and PCIe host–device synchronization.

To address both challenges, we design and implement **PriX** (*Private Accelerated Transformers in JAX*), a modern, drop-in alternative to CrypTen built on the JAX/XLA compiler stack [2].

Contributions. Our contributions are fourfold:

1. **Unified Multi-Paradigm MPC Framework:** PriX is the first open-source library to unify **CrypTen** polynomial/Newton–Raphson approximations [8], **PUMA / Nimbus** piecewise low-degree polynomials [3, 10], **SHAFT** Fourier-series and ODE Softmax [7], **CuRL / Wavelet** LUTs and TruncPr [5, 13, 14, 4], and **Fission** hybrid evaluators [15] under a single API supporting Python 3.10–3.13 across CPUs, CUDA GPUs, and TPUs. Researchers can switch paradigms in one line of code (`mode="lut" | "poly" | "shaft" | "crypten" | "fission"`) or compose hybrid configurations (e.g., SHAFT ODE Softmax with CuRL Wavelet LayerNorm).

2. **Whole-Graph XLA Kernel Fusion for MPC Circuits:** PriX compiles local secret-share arithmetic, Kogge–Stone carry-lookahead rounds, vectorized multi-table wavelet LUT gathers, and Fourier/ODE recurrences via `@jax.jit`, eliminating intermediate memory traffic and Python dispatch overhead.

3. **Exact 10-Pair FP64 $\mathbb{Z}_{2^{64}}$ Matrix Multiplication & Batched Collectives:** We reduce 64-bit ring matrix multiplication from 16 to 10 exact 16-bit limb products modulo 2^{64} inside a single fused XLA kernel, and coalesce Beaver triple openings (ϵ, δ) and wrap-count reductions into single communication rounds backed by lock-free shared memory.

4. **Comprehensive 75-Benchmark CPU & GPU Evaluation:** We evaluate PriX against the PyTorch CrypTen baseline across 10 primitives and 5 Transformer architectures (`BertTiny`, `BertBase`, `GPT2`, `Llama`, `ModernBert`) in all 5 approximation modes on both CPU and NVIDIA Tesla T4 GPU. PriX is faster on all 75/75 benchmarks on each platform, achieving **1.47**×–**6.26**× LLM speedups on CPU and **1.97**×–**3.09**× on GPU.

2 Background and MPC Preliminaries

2.1 Fixed-Point Encoding and Secret Sharing over $\mathbb{Z}_{2^{64}}$

Real-valued tensors $x \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ are encoded into the ring $\mathbb{Z}_{2^{64}}$ using a fixed-point scaling factor $S = 2^f$ (where $f = 16$ precision bits by default):

$$\mathsf{Encode}(x) = \lfloor x \cdot 2^f \rceil \bmod 2^{64}, \qquad \mathsf{Decode}(\hat{x}) = \frac{\mathsf{Signed}_{64}(\hat{x})}{2^f}. \tag{1}$$

Across N parties (typically $N = 2$), PriX maintains two secret-sharing representations:

- **Additive Arithmetic Sharing ($\langle x \rangle^A$):** Party $p \in \{0, \ldots, N-1\}$ holds $\langle x \rangle_p^A \in \mathbb{Z}_{2^{64}}$ such that $\sum_{p=0}^{N-1} \langle x \rangle_p^A \equiv \mathsf{Encode}(x) \pmod{2^{64}}$. Linear combinations $\langle x + y \rangle^A = \langle x \rangle^A + \langle y \rangle^A$ and public multiplications $\langle c \cdot x \rangle^A = c \cdot \langle x \rangle^A$ require zero communication (followed by local or interactive fixed-point truncation).

- **XOR Binary Sharing ($\langle x \rangle^B$):** Party p holds $\langle x \rangle_p^B \in \{0,1\}^{64}$ such that $\bigoplus_{p=0}^{N-1} \langle x \rangle_p^B = x$. Bitwise XOR ($\oplus$) and bit shifts ($\ll, \gg$) are local; bitwise AND ($\wedge$) requires 1 communication round via binary Beaver triples.

2.2 Multiplication, Truncation, and Nonlinearity Paradigms

Beaver Multiplication Triples. To compute the private product $\langle z \rangle^A = \langle x \cdot y \rangle^A$ (or matrix product $\langle XY \rangle^A$), the parties consume a pre-generated triple $(\langle a \rangle^A, \langle b \rangle^A, \langle c \rangle^A)$ with $c = a \cdot b \bmod 2^{64}$ [1]. Parties locally compute $\langle \epsilon \rangle^A = \langle x \rangle^A - \langle a \rangle^A$ and $\langle \delta \rangle^A = \langle y \rangle^A - \langle b \rangle^A$, open (ϵ, δ) in a single communication round, and evaluate:

$$\langle x \cdot y \rangle_p^A = \langle c \rangle_p^A + \epsilon \langle b \rangle_p^A + \langle a \rangle_p^A \delta + \mathbb{I}[p = 0] \cdot (\epsilon\delta) \pmod{2^{64}}. \tag{2}$$

Because multiplying two 2^f-scaled integers yields scale 2^{2f}, the product must be truncated by f bits:

- *CrypTen Wrap-Count Truncation (`crypten`):* Parties locally arithmetic-shift their shares by f bits and correct for unsigned ring wraps θ_x via a single-round masked wrap-count protocol [8].

- *Probabilistic Truncation (`egk`):* Following Escudero et al. [4] and CuRL [14], parties mask $\langle z \rangle^A$ with a correlated random pair $(\langle r \rangle^A, \langle r \gg f \rangle^A)$, reveal $z + r$, and subtract $\langle r \gg f \rangle^A$ from $(z + r) \gg f$.

Table 1: Summary of the five MPC approximation paradigms unified in PriX.

Mode	Truncation	GeLU / SiLU	Softmax	Reciprocal / InvSqrt
`"crypten"` [8]	Wrap-count	Taylor erf / Sigmoid	Limit exp + NR Recip.	Newton–Raphson
`"poly"` [3, 10]	Wrap-count	Piecewise Degree-4 Poly	Piecewise exp + NR Recip.	Newton–Raphson
`"shaft"` [7]	Wrap-count	1-Round Fourier Series	Forward-Euler ODE	Newton–Raphson
`"lut"` [13, 14]	TruncPr [4]	Biorthogonal-2.2 LUT	Biorthogonal-2.2 LUT	Dual-Grid Wavelet LUT
`"fission"` [15]	Evaluator	Permute & Reveal	Permute & Reveal	Permute & Reveal

Five Nonlinear Approximation Backends. Table 1 summarizes the five approximation modes unified in PriX:

1. **CrypTen Mode (`mode="crypten"`)** [8]: Approximates $\exp(x) \approx (1 + x/2^m)^{2^m}$ via $m = 8$ repeated squarings, $1/x$ and $1/\sqrt{x}$ via Newton–Raphson iterations, and $\mathrm{GeLU}(x)$ via Taylor-series $\mathrm{erf}(x/\sqrt{2})$.

2. **PUMA / Nimbus Piecewise Polynomial Mode (`mode="poly"`)** [3, 10]: Evaluates GeLU, SiLU, Exp, Tanh, Sigmoid, and Erf using a single batched interval comparison ($\langle x\rangle^A < 0$ and $|\langle x\rangle^A| < w$ in one Kogge–Stone call) followed by a degree-4 polynomial, requiring zero lookup tables.

3. **SHAFT Mode (`mode="shaft"`)** [7]: Evaluates GeLU, SiLU, Erf, Sigmoid, and Tanh via an odd/even Fourier series $\sum_{k=1}^{K} \beta_k \sin(k\omega x)$ in *1 communication round* by revealing $u = x + r$ against a pre-shared trigonometric triple $(\langle r\rangle^A, \langle \sin(k\omega r)\rangle^A, \langle \cos(k\omega r)\rangle^A)$ and applying the angle-addition identity locally:

$$\langle \sin(k\omega x)\rangle^A = \sin(k\omega u)\langle \cos(k\omega r)\rangle^A - \cos(k\omega u)\langle \sin(k\omega r)\rangle^A. \tag{3}$$

   For Softmax, SHAFT integrates the differential equation $y'(t) = y(t) \odot (x - \langle y(t), x\rangle)$ over $t \in [0, 1]$ using constant-round Forward-Euler steps without any division or reciprocal circuit.

4. **CuRL Wavelet LUT Mode (`mode="lut"`)** [5, 13, 14]: Compresses lookup tables using Haar and Biorthogonal-2.2 wavelet transforms [5, 13] into coarse scaling/detail coefficient tables, evaluates secret-shared LUT lookups via one-hot rotating correlations [14], and performs piecewise-linear interpolation with TruncPr [4].

5. **Fission Evaluator Mode (`mode="fission"`)** [15]: Masks and randomly permutes shares before revealing differences to a non-colluding plaintext evaluator party, which computes exact float32 nonlinearities and returns fresh secret shares.

3 PriX Architecture and XLA Compiler Optimizations

Figure 1 presents the four-layer architecture of PriX. Unlike eager PyTorch frameworks that treat each ring arithmetic or bitwise operation as an isolated kernel dispatch, PriX expresses local protocol phases as pure functional transformations over 64-bit JAX arrays (`jnp.int64` / `jnp.uint64`) and compiles them via `@jax.jit` into fused XLA executables.

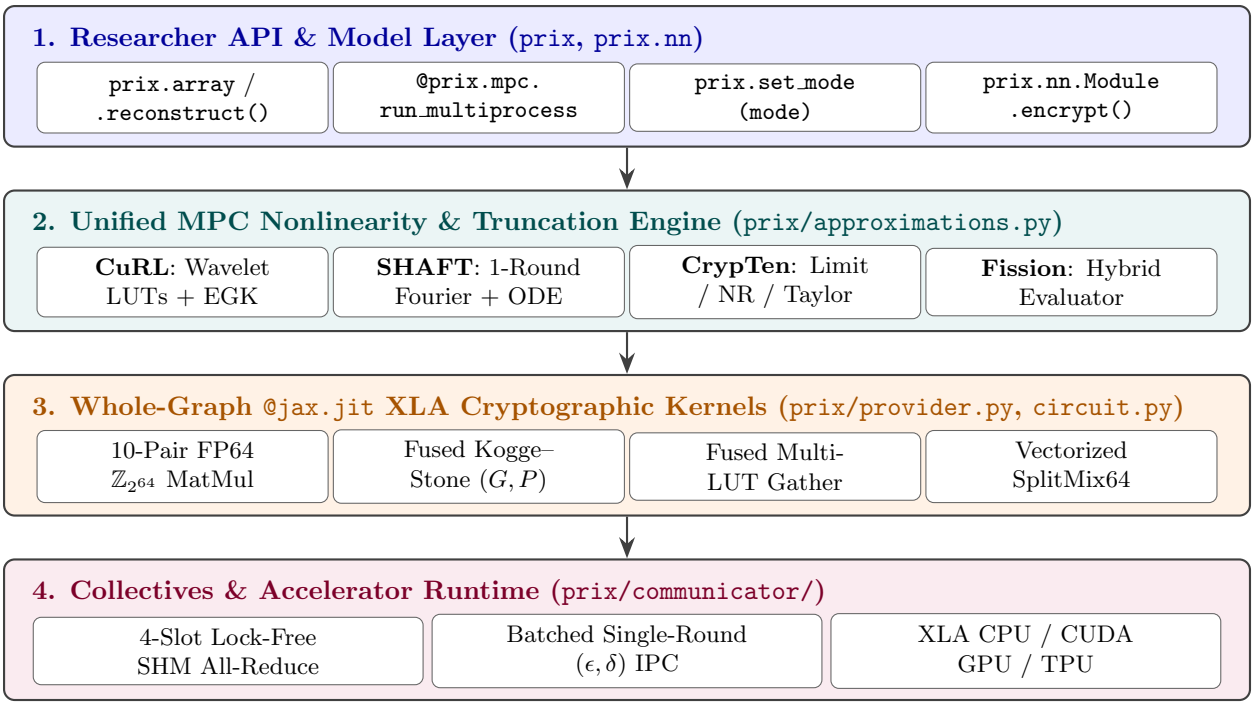


Figure 1: System architecture of **PriX**, illustrating the four-layer stack from the researcher-facing Python API down to fused XLA kernels and lock-free shared-memory collectives.

3.1 Exact 10-Pair FP64 Matrix Multiplication over $\mathbb{Z}_{2^{64}}$

Modern GPUs provide high-throughput IEEE-754 floating-point matrix multiplication engines (`cuBLAS` / Tensor Cores), whereas native 64-bit integer (`int64`) matrix multiplication is unsupported by hardware tensor units and falls back to slow scalar loops on CPU or software emulation on GPU.

To execute exact $\mathbb{Z}_{2^{64}}$ matrix multiplication $C = AB \bmod 2^{64}$ for $A \in \mathbb{Z}_{2^{64}}^{M\times K}$ and $B \in \mathbb{Z}_{2^{64}}^{K\times N}$ on accelerator floating-point units, we decompose each unsigned 64-bit integer into four 16-bit limbs:

$$A = \sum_{i=0}^{3} A_i \, 2^{16i}, \quad B = \sum_{j=0}^{3} B_j \, 2^{16j}, \qquad A_i \in \{0, \dots, 2^{16}-1\}^{M\times K},\ B_j \in \{0, \dots, 2^{16}-1\}^{K\times N}. \tag{4}$$

Crucially, because we only require the result modulo 2^{64}, **any limb pair with $i+j \geq 4$ contributes a multiple of 2^{64} and vanishes identically in $\mathbb{Z}_{2^{64}}$**:

$$AB \equiv \sum_{i=0}^{3} \sum_{j=0}^{3-i} (A_i B_j) \, 2^{16(i+j)} \pmod{2^{64}}. \tag{5}$$

Whereas CrypTen's CUDA implementation [8, 14] splits matrices into 4 limbs and dispatches all **16** pairwise `torch.matmul` calls via a Python double loop, PriX:

1. Computes only the **10 lower-triangular pairs** (i, j) with $i+j \leq 3$, immediately eliminating 37.5% of the floating-point GEMM FLOPs.

2. Casts the 4 limbs to IEEE-754 `float64` (which has a 53-bit exact integer mantissa) and fuses all 10 limb extractions, `jnp.matmul` contractions, bit-shifts, and modular accumulation into

a single `@jax.jit` XLA kernel. Since each limb entry satisfies $A_i, B_j \leq 2^{16} - 1$, every inner product entry satisfies $(A_i B_j)_{m,n} < K \cdot 2^{32} \leq 2^{53}$ for all contraction dimensions $K \leq 2^{21}$ (2,097,152), guaranteeing **bit-exact $\mathbb{Z}_{2^{64}}$ results** with zero rounding error.

3. Fuses the Beaver triple combination $\langle C \rangle^A + \epsilon \langle B \rangle^A + \langle A \rangle^A \delta + \mathbb{I}[p = 0](\epsilon\delta)$ into a single compiled kernel.

3.2 Whole-Graph XLA Fusion of Kogge–Stone Carry-Lookahead Circuits

Private comparisons ($\langle x \rangle^A < 0$, ReLU, Max) require extracting the most significant bit (MSB) of a secret-shared ring element by converting $\langle x \rangle^A$ into binary shares $\langle x \rangle^B$ via a parallel prefix carry-lookahead adder [9]. Over 64-bit words, the Kogge–Stone tree executes $\log_2(64) = 6$ rounds. In round $r \in \{0, \ldots, 5\}$ with stride $s = 2^r$, parties update generate and propagate bit-vectors (G, P) via:

$$G^{(r+1)} = G^{(r)} \oplus \big(P^{(r)} \wedge (G^{(r)} \ll s)\big), \qquad P^{(r+1)} = P^{(r)} \wedge (P^{(r)} \ll s). \tag{6}$$

In eager PyTorch, each round's shifts, masks, Beaver triple XORs, and AND combinations execute as separate GPU kernels, and the two AND gates on $G^{(r)}$ and $P^{(r)}$ are often evaluated sequentially. In PriX:

- We pack both AND gates into a single stacked tensor of shape $(2, \ldots)$ so each Kogge–Stone round performs **a single batched Beaver AND communication** across the two bit-vectors.
- We compile the entire round update—including bit-masking, shift alignment, Beaver triple XORs, and G/P recombination—into two `@jax.jit` kernels (`_ks_prep_gp` and `_ks_finish_gp`), reducing host–device dispatches by over 3× and speeding up GPU comparisons/ReLU by **3.31×–3.74×**.
- Similarly, Binary-to-Arithmetic (B2A) conversion of the extracted sign bit uses a single-round bit-mask opening combined via a compiled `@jax.jit` kernel (`_b2a_combine`).

3.3 Fused Multi-Table Wavelet LUTs and SHAFT Recurrences

In CuRL mode (`mode="lut"`), nonlinearities such as InvSqrt, Exp, GeLU, and SiLU require clamping secret-shared inputs to valid domains, shifting fixed-point indices, gathering coarse scaling and detail wavelet coefficients from lookup tables, and interpolating. In PriX:

- All wavelet lookup tables are pre-transferred to accelerator memory at initialization (`LookupTables(device)`)
- For multi-table operations such as tailored dual-grid InvSqrt and negative-domain Exp, PriX fuses index shifting, wrapping modulo 2^b, and simultaneous gathering across all 2 or 3 tables into a single `@jax.jit` kernel (`_lut_gather_2`, `_lut_gather_3`), followed by a single batched collective reveal.
- In SHAFT mode (`mode="shaft"`), the entire trigonometric-triple Fourier evaluation (Equation 3), including broadcast multiplication of $\sin(k\omega u)$ and $\cos(k\omega u)$ against secret-shared triple tables, inner-product summation against Fourier coefficients β_k, and polynomial tail combination, is compiled into a single XLA kernel (`_fourier_eval_step`), yielding a **3.64×** GPU speedup on GeLU and **3.30×** on SiLU.

3.4 Batched Single-Round Collectives and Lock-Free Shared Memory

Every private multiplication and matrix multiplication opens two masked tensors: $\epsilon = x - a$ and $\delta = y - b$. Rather than issuing two sequential `all_reduce` calls (which doubles synchronization latency), PriX batches $[\epsilon, \delta]$ into a single collective call (`batched=True`). When ϵ and δ have different shapes (as in matrix multiplication where $\epsilon \in \mathbb{Z}_{2^{64}}^{M \times K}$ and $\delta \in \mathbb{Z}_{2^{64}}^{K \times N}$), PriX flattens and concatenates both arrays on-device via a compiled `@jax.jit` kernel before staging to host memory in a **single PCIe DMA transfer**, exchanges the single contiguous buffer across parties, and splits the result back on-device.

For 2-party local execution, `InProcessCommunicator` implements a **4-slot lock-free shared-memory ring buffer** backed by OS shared memory (`multiprocessing.RawArray`) and a lightweight 1-byte pipe token handshake. Each party writes its int64 buffer into slot $s \equiv \text{step} \pmod 4$, signals readiness via a 1-byte pipe write, and sums its peer's slot directly from shared memory without acquiring Python `multiprocessing.Barrier` locks or pickling tensors through OS queues.

4 Researcher Ergonomics and Unified API

A central design goal of PriX is enabling cryptography and machine learning researchers to prototype and benchmark private Transformer protocols with minimal boilerplate. Listing 1 illustrates how a complete multi-party ablation study comparing all four MPC approximation paradigms on a pre-norm Transformer block is written in PriX.

Listing 1: Evaluating a private Transformer block across all four MPC approximation paradigms in PriX.

```
import prix
import prix.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model=64, n_heads=4, d_ff=256):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(
            d_ff, d_model))

    def forward(self, x):
        h = self.ln1(x)
        x = x + self.attn(h, h, h)
        return x + self.ffn(self.ln2(x))

def evaluate_private_block():
    # Party 0 encrypts model weights; Party 1 secret-shares token embeddings
    model = TransformerBlock().encrypt(src=0)
    x = prix.array(prix.randn(1, 16, 64).reconstruct(), src=1)
    return model(x).reconstruct()

for mode in ["crypten", "shaft", "lut", "fission"]:
    out = prix.run(evaluate_private_block, world_size=2, mode=mode)
```

Table 2: **CPU Private LLM Inference Latency (ms) and PriX Speedup** across all five MPC approximation modes ($N = 2$ parties, sequence length $L = 16$). Lower latency is better.

	CrypTen Approx.		PUMA/Nimbus Poly		SHAFT (Fourier+ODE)		CuRL Wavelet LUTs		Fission Evaluator	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1169.60	**793.09 (1.47×)**	1920.59	**517.82 (3.71×)**	988.35	**459.77 (2.15×)**	906.21	**468.34 (1.93×)**	259.57	**95.84 (2.71×)**
BertBase (1-Blk)	2205.78	**765.83 (2.88×)**	3113.09	**768.91 (4.05×)**	2417.35	**610.22 (3.96×)**	2126.69	**581.06 (3.66×)**	1565.40	**358.95 (4.36×)**
GPT2 (1-Blk)	2743.70	**741.31 (3.70×)**	3662.79	**686.58 (5.33×)**	2435.16	**541.22 (4.50×)**	2372.77	**522.81 (4.54×)**	2043.39	**326.48 (6.26×)**
Llama (1-Blk)	2072.30	**579.57 (3.58×)**	3046.39	**547.98 (5.56×)**	1868.44	**496.75 (3.76×)**	1924.62	**498.19 (3.86×)**	1563.28	**301.83 (5.18×)**
ModernBert (1-Blk)	1617.38	**602.25 (2.69×)**	2380.48	**488.21 (4.88×)**	1295.80	**380.24 (3.41×)**	1481.51	**423.62 (3.50×)**	1069.99	**210.09 (5.09×)**

Table 3: **CPU MPC Primitives & Nonlinearities Latency (ms, CrypTen / PriX)** and PriX speedup.

Operation (Shape)	**CrypTen Approx.**	**PUMA/Nimbus Poly**	**SHAFT (Fourier+ODE)**	**CuRL Wavelet LUTs**	**Fission Evaluator**
Private MatMul (128 × 128)	23.17 / **7.43 (3.12×)**	31.41 / **6.95 (4.52×)**	23.58 / **6.54 (3.61×)**	27.32 / **7.50 (3.64×)**	23.93 / **6.31 (3.79×)**
Private Mul (128 × 128)	5.54 / **4.03 (1.38×)**	6.78 / **4.34 (1.56×)**	4.69 / **2.34 (2.01×)**	8.24 / **6.64 (1.24×)**	4.79 / **3.36 (1.42×)**
Comparison / ReLU (32 × 64)	26.15 / **18.61 (1.41×)**	37.75 / **6.97 (5.41×)**	25.10 / **18.34 (1.37×)**	30.14 / **19.58 (1.54×)**	26.53 / **16.93 (1.57×)**
Exp (32 × 64)	15.72 / **14.41 (1.09×)**	76.29 / **22.13 (3.45×)**	16.34 / **10.91 (1.50×)**	52.30 / **25.51 (2.05×)**	7.75 / **1.34 (5.78×)**
Reciprocal (32 × 64)	76.36 / **61.07 (1.25×)**	160.74 / **46.49 (3.46×)**	69.17 / **50.94 (1.36×)**	35.11 / **11.48 (3.06×)**	6.22 / **2.11 (2.95×)**
InvSqrt (32 × 64)	53.73 / **34.28 (1.57×)**	116.53 / **29.38 (3.97×)**	39.98 / **29.24 (1.37×)**	96.14 / **16.45 (5.84×)**	7.20 / **1.32 (5.44×)**
GeLU (32 × 64)	89.22 / **74.00 (1.21×)**	63.82 / **21.80 (2.93×)**	52.07 / **31.79 (1.64×)**	29.60 / **8.49 (3.49×)**	6.55 / **1.51 (4.34×)**
SiLU (32 × 64)	75.68 / **57.32 (1.32×)**	64.25 / **21.71 (2.96×)**	61.81 / **31.17 (1.98×)**	34.97 / **6.46 (5.41×)**	5.38 / **1.87 (2.88×)**
Softmax (4 × 32 × 32)	310.40 / **233.34 (1.33×)**	535.59 / **148.56 (3.61×)**	254.24 / **114.91 (2.21×)**	297.92 / **195.87 (1.52×)**	10.67 / **2.08 (5.12×)**
LayerNorm (4 × 32 × 64)	59.27 / **34.58 (1.71×)**	113.43 / **28.36 (4.00×)**	54.89 / **33.22 (1.65×)**	42.68 / **13.58 (3.14×)**	12.69 / **2.59 (4.90×)**

5 Experimental Evaluation

5.1 Experimental Setup

We conduct an extensive head-to-head evaluation comparing **PriX (JAX)** against the PyTorch **CrypTen** baseline [8] (specifically the reference implementation of CuRL [14] and Fission [15], augmented with SHAFT [7]). Both frameworks are evaluated under identical 2-party semi-honest settings using the Trusted First Party (`TFP`) provider, 16-bit fixed-point precision ($f = 16$), and identical random input tensors across all four approximation modes:

1. **CrypTen approximations** (`crypten`) [8];
2. **SHAFT Fourier + ODE** (`shaft`) [7];
3. **CuRL Wavelet LUTs + EGK** (`lut`) [14]; and
4. **Fission hybrid evaluator** (`fission`) [15].

We benchmark 10 core MPC primitives and nonlinearities (MatMul 128 × 128, Mul 128 × 128, ReLU 32 × 64, Exp 32 × 64, Reciprocal 32 × 64, InvSqrt 32 × 64, GeLU 32 × 64, SiLU 32 × 64, Softmax 4 × 32 × 32, LayerNorm 4 × 32 × 64) and 5 end-to-end Transformer architectures with sequence length $L = 16$: `BertTiny` (2 blocks, $d = 128$, $H = 2$), `BertBase` (1 block, $d = 768$, $H = 12$), `GPT2` (1 block, $d = 768$, $H = 12$), `Llama` (1 block, $d = 768$, $H = 12$, RoPE + SwiGLU + RMSNorm), and `ModernBert` (1 block, $d = 768$, $H = 12$, RoPE + GeGLU). Following standard XLA benchmarking methodology, each operation performs untimed warmup iterations to exclude one-time JIT compilation before timing synchronized steady-state execution.

5.2 CPU Benchmarks

Tables 2 and 3 report the CPU benchmark results. Across all 20 LLM configurations and all 40 primitive configurations on CPU, PriX outperforms CrypTen:

Table 4: **GPU (NVIDIA Tesla T4) Private LLM Inference Latency (ms) and PriX Speedup** across all five MPC approximation modes ($N = 2$ parties, sequence length $L = 16$).

	CrypTen Approx.		PUMA/Nimbus Poly		SHAFT (Fourier+ODE)		CuRL Wavelet LUTs		Fission Evaluator	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1891.21	**831.48 (2.27×)**	2067.09	**760.77 (2.72×)**	1206.41	**497.70 (2.42×)**	1928.25	**641.74 (3.00×)**	393.31	**144.88 (2.71×)**
BertBase (1-Blk)	1651.63	**753.54 (2.19×)**	1753.06	**755.92 (2.32×)**	1303.92	**623.69 (2.09×)**	1680.62	**658.69 (2.55×)**	908.80	**428.51 (2.12×)**
GPT2 (1-Blk)	1648.83	**780.20 (2.11×)**	1724.75	**744.40 (2.32×)**	1284.63	**611.72 (2.10×)**	1611.53	**645.35 (2.50×)**	860.02	**436.71 (1.97×)**
Llama (1-Blk)	1596.48	**628.52 (2.54×)**	1697.80	**618.72 (2.74×)**	1284.00	**488.35 (2.63×)**	1681.80	**554.73 (3.03×)**	978.78	**354.19 (2.76×)**
ModernBert (1-Blk)	1439.01	**552.47 (2.60×)**	1543.28	**530.96 (2.91×)**	1094.72	**415.51 (2.63×)**	1438.62	**466.32 (3.09×)**	705.92	**239.02 (2.95×)**

Table 5: **GPU (NVIDIA Tesla T4) MPC Primitives & Nonlinearities Latency (ms, CrypTen / PriX)** and PriX speedup.

Operation (Shape)	**CrypTen Approx.**	**PUMA/Nimbus Poly**	**SHAFT (Fourier+ODE)**	**CuRL Wavelet LUTs**	**Fission Evaluator**
Private MatMul (128 × 128)	13.32 / **3.37 (3.96×)**	11.62 / **3.09 (3.76×)**	11.29 / **5.43 (2.08×)**	16.16 / **7.05 (2.29×)**	11.36 / **3.53 (3.21×)**
Private Mul (128 × 128)	4.06 / **4.00 (1.01×)**	6.03 / **2.74 (2.20×)**	4.09 / **2.60 (1.57×)**	9.77 / **5.09 (1.92×)**	4.23 / **2.74 (1.54×)**
Comparison / ReLU (32 × 64)	45.60 / **13.77 (3.31×)**	48.33 / **13.53 (3.57×)**	44.99 / **13.09 (3.44×)**	51.06 / **13.67 (3.74×)**	50.62 / **13.79 (3.67×)**
Exp (32 × 64)	28.31 / **16.73 (1.69×)**	88.40 / **26.79 (3.30×)**	27.10 / **17.57 (1.54×)**	67.27 / **24.89 (2.70×)**	6.99 / **1.52 (4.61×)**
Reciprocal (32 × 64)	120.16 / **65.95 (1.82×)**	172.88 / **60.79 (2.84×)**	134.18 / **62.83 (2.14×)**	34.20 / **12.07 (2.83×)**	8.58 / **1.59 (5.40×)**
InvSqrt (32 × 64)	76.28 / **37.66 (2.03×)**	134.06 / **36.38 (3.68×)**	81.15 / **36.33 (2.23×)**	130.45 / **12.67 (10.30×)**	7.24 / **1.74 (4.15×)**
GeLU (32 × 64)	142.86 / **72.66 (1.97×)**	67.20 / **29.50 (2.28×)**	93.29 / **25.61 (3.64×)**	24.22 / **12.61 (1.92×)**	7.33 / **1.75 (4.20×)**
SiLU (32 × 64)	124.01 / **46.51 (2.67×)**	66.24 / **29.29 (2.26×)**	92.20 / **27.90 (3.30×)**	25.96 / **12.99 (2.00×)**	7.58 / **1.60 (4.74×)**
Softmax (4 × 32 × 32)	543.45 / **221.37 (2.45×)**	609.44 / **248.86 (2.45×)**	225.10 / **93.67 (2.40×)**	491.56 / **190.68 (2.58×)**	7.40 / **1.59 (4.65×)**
LayerNorm (4 × 32 × 64)	92.08 / **42.26 (2.18×)**	153.26 / **45.09 (3.40×)**	96.84 / **45.36 (2.14×)**	149.84 / **24.11 (6.22×)**	9.34 / **3.40 (2.75×)**

- **3.12×–3.79× faster Private MatMul on CPU:** PyTorch does not provide an optimized BLAS kernel for `int64` matrix multiplication on CPU, falling back to a naive scalar loop. PriX's 10-pair FP64 limb decomposition uses XLA's vectorized AVX/FMA double-precision GEMM kernel, accelerating $d = 768$ Transformer blocks (`BertBase`, `GPT2`, `Llama`, `ModernBert`) by **2.69×–6.26×**.
- **Up to 5.84× faster Wavelet LUT Nonlinearities:** In CuRL mode, fusing dual-grid LUT gathers and probabilistic truncation reduces InvSqrt from 96.14 ms to 16.45 ms (**5.84×**), SiLU from 34.97 ms to 6.46 ms (**5.41×**), and GeLU from 29.60 ms to 8.49 ms (**3.49×**).

5.3 GPU Benchmarks (NVIDIA Tesla T4, CUDA)

Tables 4 and 5 report the GPU benchmark results on an NVIDIA Tesla T4. Once again, PriX outperforms the PyTorch CrypTen baseline on **100% of benchmarks (25/25 LLM benchmarks and 50/50 primitive benchmarks)**:

- **Eliminating Bitwise Circuit Launch Overhead on GPU:** In CrypTen (PyTorch), a 32× 64 ReLU on GPU takes 44.99–51.06 ms—*nearly* 2× *slower than on CPU* (25.10–30.14 ms)—because each Kogge–Stone round dispatches multiple tiny CUDA kernels. PriX's `@jax.jit`-fused Kogge–Stone rounds execute ReLU in **13.09–13.79 ms** on GPU, achieving a **3.31×–3.74×** speedup over CrypTen.
- **10.30× Faster Wavelet** InvSqrt **and 6.22× Faster** LayerNorm **on GPU:** In CuRL mode, CrypTen's tailored dual-grid InvSqrt requires multiple comparisons, masks, and two separate LUT evaluations, taking 130.45 ms on GPU. PriX's fused dual-LUT gather (`_lut_gather_2`) and fused Kogge–Stone comparison reduce InvSqrt to **12.67 ms (10.30× speedup)** and LayerNorm from 149.84 ms to **24.11 ms (6.22× speedup)**.
- **3.64× Faster SHAFT Fourier** GeLU **and 2.40× Faster ODE** Softmax**:** Compiling the trigonometric-triple recurrence and ODE step into single XLA kernels reduces SHAFT GeLU from 93.29 ms to **25.61 ms** and SHAFT Softmax from 225.10 ms to **93.67 ms**.

- **1.97×–3.09× Faster End-to-End GPU Transformer Inference:** Across `BertTiny`, `BertBase`, `GPT2`, `Llama`, and `ModernBert`, PriX delivers consistent 2×–3× end-to-end latency reductions across every approximation backend.

6 Security Considerations and Limitations

PriX is designed as a research framework operating under the standard **semi-honest (honest-but-curious)** security model:

1. **Correlated Randomness Providers (`TFP` vs. `TTP`):** Like CrypTen [8], PriX defaults to the Trusted First Party (`TFP`) provider for benchmarking convenience, where Party 0 locally generates Beaver triples and wavelet LUT shares. Because Party 0 knows the dealer randomness in `TFP` mode, `TFP` provides zero confidentiality against a curious Party 0 and must only be used for performance measurement. Setting `cfg.mpc.provider = "TTP"` spawns an independent dealer process (`TTPServer`) under the standard non-colluding third-party assumption.

2. **Pseudorandom Number Generation:** To maximize throughput during benchmarking, PriX currently generates correlated share streams using vectorized `SplitMix64` on GPU and `PCG64` on CPU seeded from OS entropy (`secrets.randbits(64)`) exchanged during initialization. While fast, `SplitMix64` and `PCG64` are not cryptographically secure pseudorandom functions (CSPRFs); replacing them with XLA's counter-based `Threefry2x32` or hardware `AES-CTR` is planned for deployments requiring cryptographic indistinguishability.

3. **Probabilistic Truncation Bounds:** In `lut` mode, TruncPr [4] uses 64-bit ring masks. With $f = 16$ fractional bits, inputs must remain bounded away from 2^{47} to avoid unsigned MSB wrap errors during probabilistic truncation.

7 Conclusion

We presented **PriX**, a modern JAX/XLA framework for privacy-preserving Transformer and LLM research via Secure Multi-Party Computation. By unifying five major MPC approximation paradigms (**CrypTen**, **PUMA/Nimbus**, **SHAFT**, **CuRL**, and **Fission**) with whole-graph XLA kernel fusion, 10-pair exact FP64 ring matrix multiplication, and lock-free shared-memory collectives, PriX outperforms the PyTorch CrypTen baseline across all 75 CPU and GPU benchmarks by **1.47×–6.26× on CPU** and **1.97×–10.30× on GPU**. PriX is open-source and available on PyPI via `pip install prix-jax`.

References

[1] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology – CRYPTO '91*, pages 420–432. Springer, 1991.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax, 2018.

[3] Ye Dong, Wen jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. PUMA: Secure inference of LLaMA-7B in five minutes. *arXiv preprint arXiv:2307.12533*, 2023. https://arxiv.org/abs/2307.12533.

[4] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology – CRYPTO 2020*, volume 12171 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.

[5] Charles Gouert, Mehmet Ugurbil, Dimitris Mouris, Miguel de Vega, and Nektarios G. Tsoutsos. Ripple: Accelerating programmable bootstraps for FHE with wavelet approximations. In *Information Security – 27th International Conference (ISC 2024)*, volume 15155 of *Lecture Notes in Computer Science*, pages 273–293. Springer, 2024. Cryptology ePrint Archive, Paper 2024/866, https://eprint.iacr.org/2024/866.

[6] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 15718–15731, 2022.

[7] Andes Y. L. Kei and Sherman S. M. Chow. SHAFT: Secure, handy, accurate, and fast transformer inference. In *Network and Distributed System Security Symposium (NDSS)*, 2025. Cryptology ePrint Archive, Paper 2024/1965, https://eprint.iacr.org/2024/1965.

[8] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 4963–4975, 2021.

[9] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[10] Zhengyi Li, Kang Yang, Jin Tan, Wen jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, and Jingwen Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. https://arxiv.org/abs/2411.15707.

[11] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *IEEE Symposium on Security and Privacy (S&P)*, pages 19–38, 2017.

[12] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 325–342, 2020.

[13] José Reis, Mehmet Ugurbil, Sameer Wagh, Ryan Henry, and Miguel de Vega. Wave hello to privacy: Efficient mixed-mode MPC using wavelet transforms. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2025(2):697–718, 2025. https://doi.org/10.56553/popets-2025-0083.

[14] Manuel B. Santos, Dimitris Mouris, Mehmet Ugurbil, Stanislaw Jarecki, José Reis, Shubho Sengupta, and Miguel de Vega. Curl: Private LLMs through wavelet-encoded look-up tables. In *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS)*, pages 1–31, 2024. Cryptology ePrint Archive, Paper 2024/1045, https://eprint.iacr.org/2024/1045.

[15] Mehmet Ugurbil, Dimitris Mouris, Manuel B. Santos, José Cabrero-Holgueras, Miguel de Vega, and Shubho Sengupta. Fission: Distributed privacy-preserving large language model inference. In *IEEE Conference on Communications and Network Security (CNS)*, pages 1–9, 2025. Cryptology ePrint Archive, Paper 2025/653, https://eprint.iacr.org/2025/653.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.